

String Handling, Indexing & Ranging:

① `s = "Python"`

`print(s[0], s[-1], s[6])`

Ans: Error. `s[0] → P` `s[-1] → n`

Because the `s[6]` does not exist, it is out of range.

② Positive Indexing:-

It access a character using its location number.

It is traversing

Eg:- `name = 'Priya'`

<code>name[0]</code>	<code>name[1]</code>
<code>'P'</code>	<code>'r'</code>

Negative Indexing:-

It is back traversing.

Eg:- `name = 'Dharshini'`

<code>name[-1]</code>	<code>name[-2]</code>
<code>'i'</code>	<code>'n'</code>

`s = "Hello"    s[-2] = 'l'`

③ It shows error when the index that is out of range in string.

In slicing it gives available character.

`s = "code"`

`print(s[10])` 'error'

`print(s[1:10])` 'ode'

④ `s = "mississippi"`

Index of first 's' = 2

last 's' = 6

⑤ `s = "abcdefg"`

-6 -5 -4 -3 -2 -1

`print(s[-1], s[-3], s[-6])`

(g, d, a)

It negative indexing it counts from backward of the string.

⑥. String slicing in python is getting a particular position of string.

`s[start:stop:step]`

start $\rightarrow$ The index where the slicing starts.

stop $\rightarrow$ The index where the slicing stops.

step $\rightarrow$ Intermediate value which is extracted.

⑦. `s = "HelloWorld"`

`Print(s[0:5])` $\rightarrow$ Hello

`Print(s[5:])` $\rightarrow$ World

`Print(s[:5])` $\rightarrow$ Hello

`Print(s[:])` $\rightarrow$ HelloWorld

⑧ In positive step the start should be smaller than stop so it returns the blank space.

9. By using negative index we can extract the last 3 char of a string using slicing.

10. `s = "abcdefgh"`

`print(s[::-1])` → `hgfedcba`

`print(s[::2])` → `aceg`

`print(s[1::2])` → `bd fh`

11. `s = "Python"`

`print(s[::-1])`

output → `nohtyp`

The step value is (-1).

12. `s = "abcdefgh"`

`print(s[::-1])` → `hgfedcba`

`print(s[::2])` → `aceg`

`print(s[6::-1])` → `gfedc`

→ The string is scanned from backward. (right to left).

⑬. `s = "a1b2c3d4e5"`

`Print(s[1:10:3])`

Output : "12345".

⑭. `s = "Python"`

`print(s[5:0:-1])` → nohty

`print(s[5::-1])` → nohtpy

`print(s[:0:-1])` → nohty

Start → end of the string

Stop → beginning of the string.

⑮. `s = "hello"`

`print(s[100::-1])` → olleh

`print(s[::-100])` → o

It automatically counting them to the string boundaries.

⑯. `s = "abcdefghij"`

`print(s[2:-2])`

'cdefgh'

17. `s = "racecar"`
`print(s[::-1])`

'racecar'

It is plindrome.

'True'

18. `s = "Hello"`

`Print(s[1:100])` → ello

`Print(s[-100:3])` → hel

`print(s[100:200])` → — (blank space)

python does not raise an error because slicing handles out of range.

19. `s = "Hello0 1 2 3 4 5 6World"`

`print(s[:6] + " Python")`

Hello Python

20. `s = "a0 1 2 3 4 5 6 7 8 9bcd0 0efghij"`

`my_slice = slice(1, 8, 2)`

`Print(s[my_slice])`

output: bdfh